

React Native Developer - Shortlisting Task Decision Brief

Question 1 — Storage choice

I selected AsyncStorage (`@react-native-async-storage/async-storage`) for this implementation. The application is scoped as a standalone, offline-first notes app with a flat data structure and no cloud synchronization requirements. Under these constraints, AsyncStorage serves as the most efficient key-value store, allowing fast integration and predictable performance without introducing unnecessary architectural layers.

I explicitly considered WatermelonDB but rejected it due to the heavy setup overhead, C++ compilation requirements, and complex schemas which are unnecessary for a simple list of independent text entries. I also evaluated React Native MMKV; while it offers superior read/write speeds, AsyncStorage is more than sufficient for lightweight text data and is less prone to build environment complexities on standard React Native setups. Lastly, I rejected SQLite because managing custom table schemas and writing raw SQL queries for a single-table prototype adds complexity without any functional benefit. I commit to AsyncStorage as it perfectly aligns with the requirements of this offline prototype.

Question 2 — The hard problem

In my current implementation, there is no remote backend or cloud synchronization mechanism. The notes are stored entirely on the local device, meaning a cross-device conflict is physically impossible.

However, if a basic synchronization mechanism were introduced to upload the local storage array to a central database, this implementation would fail silently. Specifically, the setup stores notes as a monolithic serialized JSON array and uses a simple `updatedAt` timestamp. If two devices edit the same note offline, the system would apply a naive "last write wins" strategy. Whichever device reconnects and pushes its state last would completely overwrite the database record, silently deleting the edits made by the first device.

To handle this cleanly, I would implement Conflict-Free Replicated Data Types (CRDTs) or a change-log based synchronization protocol like the one natively provided by WatermelonDB. This would track and merge edits at a granular, character-by-character level rather than replacing the entire note. Implementing this solution would require replacing AsyncStorage with a relational offline-first engine like SQLite, which would significantly increase the application's complexity, bundle size, and development time.

Question 3 — First failure mode

In a production environment with real users, the first critical component to fail would be the performance of the notes listing page. The symptom would be noticeable lag, dropped frames during scrolling, and potentially application crashes on low-end devices when loading or saving notes.

This failure is caused by the monolithic storage design where all notes are serialized and stored under a single AsyncStorage key. As users accumulate hundreds of notes containing long paragraphs, the application must parse and stringify the entire database on the JavaScript thread during every read and write operation, blocking the UI.

The fix is to normalize the database structure by storing a lightweight index of note metadata under one key and storing each note's actual text content under its own separate key. Alternatively, migrating the storage to an SQLite database with native pagination using SQL limit and offset parameters would prevent loading the entire dataset into memory at startup.